

# UD4 – React – básicos II

---

DAW2 - DEWC

# Listas

---

## Dos aproximaciones

- Propiedad “children” -> parecido al innerHTML
- Iterar colección de datos -> con .map genero un hijo para cada elemento de la colección.

## children

```
<ListaChildren ciclo="Informática">
  <li>Diseño de interfaces web - 5 horas</li>
  <li>Desarrollo web en entorno cliente - 9 horas</li>
  <li>Despliegue de aplicaciones web - 4 horas</li>
</ListaChildren>
```

```
function ListaChildren({ ciclo, children }) {
  return (
    <div>
      <h1>Ciclo {ciclo} - Con Children</h1>
      <ul>{children}</ul>
    </div>
  );
}
```

# Listas a partir de una colección

---

Dentro del return tenemos otro return por cada elemento.

Iteramos con .map

Debemos indicar la propiedad “key”

Hook “useId” -> genera un identificador único para el componente.

```
function ListaArray({ ciclo, módulos }) {  
  const componenteId = useId();  
  return (  
    <div>  
      <h1>Ciclo {ciclo} - Por propiedad</h1>  
      <ul>  
        {módulos.map((modulo, indice) => {  
          return (  
            <li key={` ${componenteId}-${indice}`}>  
              {`${modulo.nombre} - ${modulo.horas}`}  
            </li>  
          );  
        })}  
      </ul>  
    </div>  
  );  
}
```

# Estilos

---

Recomendación => Maquetar con clases, propiedad “*className*”

Posibilidades:

- Hoja de estilos general
- Hoja de estilos por componente
- Estilos en línea, propiedad “*style*”
- CSS Module

También se pueden emplear frameworks CSS => instalar con “*npm*”

CUIDADIN -> el maquetado tradicional facilita que se pisen los estilos.

# Estilos – “style”

---

Permite asignar estilos en línea.

La propiedad “**style**” recibe un JSON con los estilos.

Cuidado con el doble {{ }}

```
<p style={{ color: "blue", fontWeight: "bold" }}>Esto es una prueba</p>
```

# Estilos – CSS Module

---

Evita conflictos entre nombres de clases CSS, hashlea los nombres de las clases de manera transparente al desarrollador.

- Definimos la hoja de estilos css con la extensión “*.module.css*”.
- Importamos asignándole un alias a la hoja de estilos.

```
import styles from './05EstiloCSSModule.module.css';
```

- Asignamos las clases a través del alias de la hoja de estilos.

```
<p className={styles.parrafo}>Esto es una prueba</p>
```

# Hook useEffect

---

Gestiona el ciclo de vida

- Montar => `[]`
- Actualizar => vacío (se dispara al renderizar), `[variables/estados]`
- Desmontar => `return () => {}`

Comando

```
useEffect(() => { /* código */ }, [/* dependencias */]);
```

# Formularios

---

Debemos gestionar el “**submit**” para no recargar la página.

Una manera de codificar consiste en:

1. Definir un estado para los JSON con los datos del formulario.
2. Asignar el valor a los “**input**” a partir del objeto estado.
3. Definir el evento “**change**” en los “input” para actualizar el objeto estado.

# Formularios - II

---

```
{modulo && (  
  <form onSubmit={handleSubmit}>  
    <input type="hidden" name="id" id="id" value={modulo.id} />  
    <label htmlFor="nombre">Nombre:</label>  
    <input  
      type="text"  
      name="nombre"  
      id="nombre"  
      value={modulo.nombre}  
      onChange={handleChange}  
    />  
      
    const handleSubmit = (e) => {  
      e.preventDefault();  
      /* const formData = new FormData(e.target); */  
      enviarDatos();  
    };  
      
    const handleChange = (e) => {  
      /* Target es de tipo input, el nombre y el valor están disponibles */  
      const { name, value } = e.target;  
      let actualizado = { ...modulo, [name]: value };  
      setModulo(actualizado);  
    };  
  )  
}
```

# Enrutado

---

Permite gestionar la navegación entre páginas.

Debemos instalar el módulo “**react-router-dom**”.

Objetos:

- ***BrowserRouter***: contexto del enrutados.
- ***Routes***: colección de rutas.
- ***Route***: asocia una ruta a un elemento (componente).
- ***Link***: genera un enlace.
- ***Outlet***: punto de renderizado en un layout.
- ***Navigate***: redirección al renderizar.
- ***useNavigate()***: redirección con JS, por algún evento.
- ***useParams()***: recupera los parámetros “*GET*” de la URL.
- ***useLocation()***: recupera información de la ruta URL. Intercambia información con propiedad “*state*”.

# Contextos globales

---

Permiten compartir información entre todos los nodos hijos.

Si lo definimos en la raíz está disponible para toda la aplicación.

## Usos comunes

- Almacenar datos de autenticación.
- Almacenar preferencias, tema, filtros, etc...
- Almacenar objetos de negocio.
- Almacenar datos de la sesión, carritos de la compra, etc...

# Contextos globales

---

**nombreContexto=createContext():** Crea un contexto.

**nombreContexto.Provider:** Define un componente que proporciona el valor del contexto a los componentes hijos.

- Define una propiedad “**value**” con un JSON que contenga los datos y métodos del contexto.
- Debe incluir la propiedad “**children**”.
- Implementar un componente envoltorio que almacene estados, implemente las funcionalidades del contexto y devuelve un JSX con el provider.

**useContext(nombreContexto):** Es un hook que permite a un componente acceder al valor del contexto.

# Optimizaciones – Carga diferida

---

## Carga diferida – React.lazy()

- React carga todos los archivos antes de comenzar la ejecución.
- Podemos definir una carga bajo demanda con “**lazy**”
- El punto de entrada es el enrutador.

```
//import AppLayout from "../components/AppLayout";  
  
const AppLayout = lazy(() => import("../components/AppLayout"));
```

## Componente “**Suspense**”

- Permite mostrar un texto alternativo durante la carga asíncrona “**lazy**”
- La propiedad “**fallback**” define el contenido alternativo.

```
<Suspense fallback={<div>Cargando Home...</div>}> <HomePage /> </Suspense>
```

# Optimizaciones – React.memo()

---

El hook memo permite cachear componentes si sus “**props**” no han cambiado.

Si el componente cambia mucho o es muy pequeño la mejora será mínima.

Ideal para elementos de listas de datos.

Envolvemos el componente a cachear al exportarlo.

```
export default memo(ListaLinea);
```

# Conclusiones

---

Léete la documentación

Prueba los ejemplos

# Preguntas

---